

Proyecto Final

Presentamos una comparación entre implementaciones secuenciales y paralelas (OpenMPI) para el cálculo de gradiente en mallas triangulares. Observamos los tiempos de ejecución y la eficiencia de la implementación paralela. Utilizamos ParaView para visualizar la solución analítica y numérica de la función $f(x, y) = 20(\sin(4x) \sin(3y) + 0.3 \cos(6x) \sin(5y) + 1)$. Usamos un Intel Core i7-4790 para la comparación de tiempos de ejecución. Encontramos que con estas especificaciones, nuestra implementación logra un pequeño speedup a la versión secuencial, con eficiencias aceptables.

Índice

1. Introducción	1
2. Cálculo del gradiente por mínimos cuadrados	1
3. Implementación secuencial	2
3.1. Lectura de datos VTK	3
3.2. Estructura de datos de triángulos	3
3.3. Solución a sistema de ecuaciones	6
3.4. Aproximación del gradiente	7
3.5. Interpolación	8
4. Implementación paralela con OpenMPI	9
5. Visualización de resultados	9
5.1. Ejemplo 1	9
5.2. Ejemplo 2	11
6. Compilación y ejecución	12
7. Comparación de tiempos de ejecución	13
8. Conclusiones	15

1. Introducción

Los modelos físicos a menudo se describen mediante ecuaciones diferenciales parciales (PDEs) que involucran funciones de varias variables. Para resolver estas PDEs, es común discretizar el dominio espacial utilizando mallas, lo que permite aproximar las soluciones numéricamente. En este proyecto, nos enfocamos en el cálculo del gradiente de una función definida en una malla triangular utilizando métodos numéricos. Los cálculos del gradiente se complican cuando se utilizan mallas irregulares, como pueden ser las triangulares. Esto requiere técnicas sofisticadas para garantizar precisión.

Supongamos que dada una malla triangular de una placa bidimensional, conocemos el valor de una función $f(x, y)$ en los centros de cada triángulo. Para la generación de la malla, utilizamos el software Gmsh, que nos permite crear mallas y exportarlas en formato VTK. Este formato nos proporciona la información necesaria sobre los nodos, triángulos y sus vecinos. Con esto, es posible realizar los cálculos del gradiente en cada uno de los nodos.

2. Cálculo del gradiente por mínimos cuadrados

Como se mencionó en la sección anterior, solo se conoce el valor de la función en los centros de los triángulos de la malla. Denotemos por el centro de un triángulo como (x_0, y_0) y $f(x_0, y_0) = f_0$. El objetivo es aproximar localmente la función f mediante un plano $p(x, y)$ de la forma

$$p(x, y) = ax + by + c$$

donde $a \approx \frac{\partial f}{\partial x}(x_0, y_0)$ y $b \approx \frac{\partial f}{\partial y}(x_0, y_0)$. Es decir, queremos determinar los coeficientes a , b y c de tal manera que el plano se ajuste lo mejor posible a los valores conocidos de la función en el centroide de cada triángulo, y el valor del centroide (x_i, y_i) de los 2 o 3 vecinos, dados por f_i para $i = 1, 2, 3$. Una

simplificación se puede realizar al pedir $p(x_0, y_0) = f_0$. Es decir, tenemos $ax_0 + by_0 + c = f_0$. Esto equivale a

$$c = f_0 - ax_0 - by_0.$$

Entonces, tenemos

$$p(x, y) = a(x - x_0) + b(y - y_0) + f_0.$$

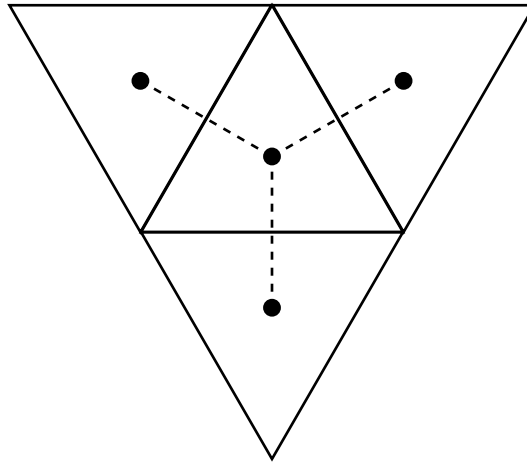


Figura 1: Ejemplo de triángulo y sus vecinos.

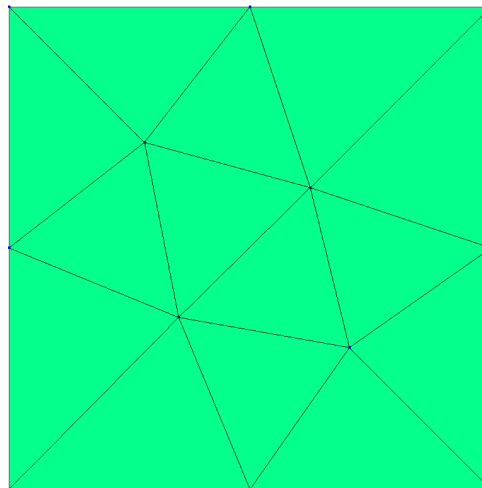


Figura 2: Ejemplo de discretización triangular. Visualización mediante Gmsh.

3. Implementación secuencial

Usaremos la [librería de VTK en C++](#) para facilitar la lectura y guardado de información. Los snippets de código que se incluyen en este documento no son la versión completa, si no un «pseudo-código» que sigue la estructura de C++.

3.1. Lectura de datos VTK

La lectura de datos se realiza en `vtk_reader.cpp`. La siguiente función realiza la lectura y convierte el contenido de `filename` en un puntero a `vtkUnstructuredGrid`. Nótese que es un smart pointer, por lo que no nos preocupamos por liberar la memoria.

```
vtkSmartPointer<vtkUnstructuredGrid> readVTKFile(const char* filename);
```

3.2. Estructura de datos de triángulos

Una vez extraída la información del VTK, podemos usar los métodos de la clase para generar la estructura de datos que representará los triángulos. En `triangle_mesh.cpp` se encuentran las implementaciones del preprocesamiento de datos, que nos permiten obtener los datos necesarios para el cálculo del gradiente en cada centroide.

```
// private attribute
std::vector<std::array<double, 3>> nodes;

// private method
void TriangleMesh::extractNodes(vtkSmartPointer<vtkUnstructuredGrid> grid)
{
    vtkPoints *points = grid->GetPoints();
    vtkIdType numPoints = points->GetNumberOfPoints();
    nodes.reserve(numPoints);

    for (vtkIdType i = 0; i < numPoints; ++i)
    {
        double p[3];
        points->GetPoint(i, p);
        nodes.push_back({p[0], p[1], p[2]});
    }
}
```

Listado 2: Extraemos los nodos que representan los vértices de los triángulos. Notemos que tenemos 3 coordenadas, ya que en general, Gmsh utiliza 3 coordenadas. En nuestro caso, solo usaremos las primeras 2, pero guardamos todas por escalabilidad.

```
// private attribute
std::vector<vtkIdType> connectivity;

// private method
void TriangleMesh::extractTriangles(vtkSmartPointer<vtkUnstructuredGrid> grid)
{
    vtkIdType totalEntities = grid->GetNumberOfCells();
    vtkNew<vtkIdList> cellPointsId;
    cellOffsets.push_back(0);

    for (vtkIdType i = 0; i < totalEntities; ++i)
    {
        if (grid->GetCellType(i) == VTK_TRIANGLE)
        {
            grid->GetCellPoints(i, cellPointsId);
            vtkIdType numIds = cellPointsId->GetNumberOfIds();

            for (vtkIdType j = 0; j < numIds; ++j)
            {
                connectivity.push_back(cellPointsId->GetId(j));
            }
            cellOffsets.push_back(connectivity.size());
            numTriangles++;
        }
    }
}
```

Listado 3: Guardamos los id's de los triángulos. Usamos el cellOffsets para determinar a cuál triángulo pertenece.

```
void TriangleMesh::buildAdjacency()
{
    std::map<std::pair<vtkIdType, vtkIdType>, std::vector<int>> edgeToCells;

    for (int cellId = 0; cellId < numTriangles; ++cellId)
    {
        vtkIdType n0 = connectivity[start];
        vtkIdType n1 = connectivity[start + 1];
        vtkIdType n2 = connectivity[start + 2];

        e1 = {std::min(n0, n1), std::max(n0, n1)};
        e2 = {std::min(n1, n2), std::max(n1, n2)};
        e3 = {std::min(n2, n0), std::max(n2, n0)};

        edgeToCells[e1].push_back(cellId);
        edgeToCells[e2].push_back(cellId);
        edgeToCells[e3].push_back(cellId);
    }

    for (const auto &[edge, cells] : edgeToCells)
    {
        if (cells.size() == 2)
        {
            cellNeighbors[cells[0]].push_back(cells[1]);
            cellNeighbors[cells[1]].push_back(cells[0]);
        }
    }
}
```

Listado 4: Generamos una matriz de adyacencia para registrar los vecinos de cada triángulo.

```
void TriangleMesh::buildNodeMap()
{
    for (int cellId = 0; cellId < numTriangles; ++cellId)
    {
        for (int j = start; j < end; ++j)
        {
            nodeToCells[connectivity[j]].push_back(cellId);
        }
    }
}
```

Listado 5: Guardamos los nodos corresponden a cada triángulo.

```
void TriangleMesh::calculateCentroids()
{
    for (int cellId = 0; cellId < numTriangles; ++cellId)
    {
        int start = cellOffsets[cellId];
        vtkIdType n0 = connectivity[start];
        vtkIdType n1 = connectivity[start + 1];
        vtkIdType n2 = connectivity[start + 2];

        double cx = (nodes[n0][0] + nodes[n1][0] + nodes[n2][0]) / 3.0;
        double cy = (nodes[n0][1] + nodes[n1][1] + nodes[n2][1]) / 3.0;
        double cz = (nodes[n0][2] + nodes[n1][2] + nodes[n2][2]) / 3.0;

        centroids[cellId] = {cx, cy, cz};
    }
}
```

Listado 6: Calculamos los centroides de cada triángulo.

3.3. Solución a sistema de ecuaciones

Escribiendo

$$\Delta x_i = x_i - x_0, \quad \Delta y_i = y_i - y_0, \quad \Delta f_i = f_i - f_0,$$

escribimos el modelo como

$$\Delta f_i \approx a \Delta x_i + b \Delta y_i.$$

Realizando una minimización de cuadrados, tenemos que el mínimo de

$$E(a, b) = \sum_{j=1}^3 (a \Delta x_j + b \Delta y_j - \Delta f_j)^2$$

se obtiene con

$$\begin{bmatrix} \sum \Delta x_j^2 & \sum \Delta x_j \Delta y_j \\ \sum \Delta x_j \Delta y_j & \sum \Delta y_j^2 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum \Delta x_j \Delta f_j \\ \sum \Delta y_j \Delta f_j \end{bmatrix}.$$

Entonces, definiendo

$$S_{xx} = \sum_{j=1}^3 \Delta x_j^2, \quad S_{yy} = \sum_{j=1}^3 \Delta y_j^2, \quad S_{xy} = \sum_{j=1}^3 \Delta x_j \Delta y_j$$
$$S_{xf} = \sum_{j=1}^3 \Delta x_j \Delta f_j, \quad S_{yf} = \sum_{j=1}^3 \Delta y_j \Delta f_j.$$

La solución es dada por

$$a = \frac{S_{yy}S_{xf} - S_{xy}S_{yf}}{D}, \quad b = \frac{S_{xx}S_{yf} - S_{xy}S_{xf}}{D}$$

donde

$$D = S_{xx}S_{yy} - S_{xy}^2$$

representa la determinante del sistema. Notemos que podemos realizar una simplificación de los valores de tal manera que obtengamos coeficientes que no dependen de f . Es decir,

$$a = \frac{S_{yy}S_x - S_{xy}S_y}{D}\Delta f, \quad b = \frac{S_{xx}S_y - S_{xy}S_x}{D}\Delta f.$$

Esto nos permite realizar una pre-computación de los coeficientes para cada triángulo sin involucrar el valor de la función en los centroides.

```
void TriangleMesh::precomputeGradientWeights()
{
    for (int cellId = 0; cellId < numTriangles; ++cellId)
    {
        auto centroid = centroids[cellId];
        auto neighbors = cellNeighbors[cellId];
        double Sxx = 0.0, Syy = 0.0, Sxy = 0.0;

        for (int neighbor : neighbors)
        {
            double Sx = centroids[neighbor][0] - centroid[0];
            double Sy = centroids[neighbor][1] - centroid[1];
            Sxx += Sx * Sx;
            Syy += Sy * Sy;
            Sxy += Sx * Sy;
        }

        double D = Sxx * Syy - Sxy * Sxy;
        for (size_t i = 0; i < neighbors.size(); ++i)
        {
            double Sx = centroids[neighbors[i]][0] - centroid[0];
            double Sy = centroids[neighbors[i]][1] - centroid[1];

            weightX[cellId][i] = (Syy * Sx - Sxy * Sy) / D;
            weightY[cellId][i] = (Sxx * Sy - Sxy * Sx) / D;
        }
    }
}
```

Listado 7: Realizamos un precálculo de los pesos de cada centroide.

3.4. Aproximación del gradiente

Con los coeficientes mencionados, obtenemos el valor del gradiente de la siguiente manera:

```
for (int cellId = 0; cellId < numTriangles; ++cellId)
{
    auto centroid = centroids[cellId];
    auto neighbors = cellNeighbors[cellId];

    double gradX = 0.0;
    double gradY = 0.0;

    for (size_t i = 0; i < neighbors.size(); ++i)
    {
        double Sf = f(centroids[neighbors[i]][0], centroids[neighbors[i]][1]) -
                    f(centroid[0], centroid[1]);

        gradX += weightX[cellId][i] * Sf;
        gradY += weightY[cellId][i] * Sf;
    }

    cellGradientsX[cellId] = gradX;
    cellGradientsY[cellId] = gradY;
}
```

Listado 8: Aproximación del gradiente en cada centroide.

3.5. Interpolación

Dado que ParaView espera los valores del gradiente en vértices de los triángulos, debemos realizar una interpolación de los valores en los centroides adyacentes.

```
for (int nodeId = 0; nodeId < mesh.getNumNodes(); ++nodeId)
{
    double sumX = 0.0;
    double sumY = 0.0;

    for (int cellId : cells)
    {
        sumX += cellGradients[cellId][0];
        sumY += cellGradients[cellId][1];
    }

    nodeGradients[nodeId][0] = sumX / numConnected;
    nodeGradients[nodeId][1] = sumY / numConnected;
}
```

Listado 9: Interpolación a los valores nodales.

4. Implementación paralela con OpenMPI

5. Visualización de resultados

5.1. Ejemplo 1

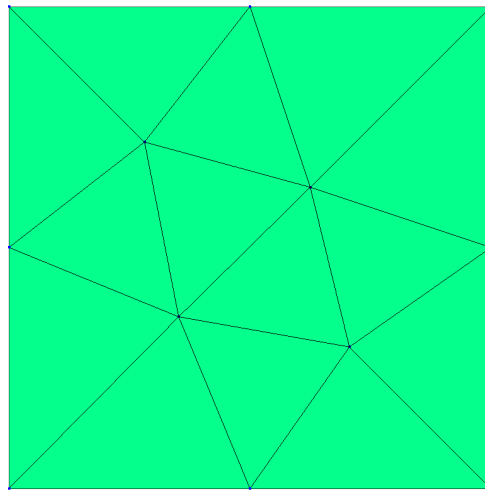


Figura 3: Ejemplo sencillo de discretización.

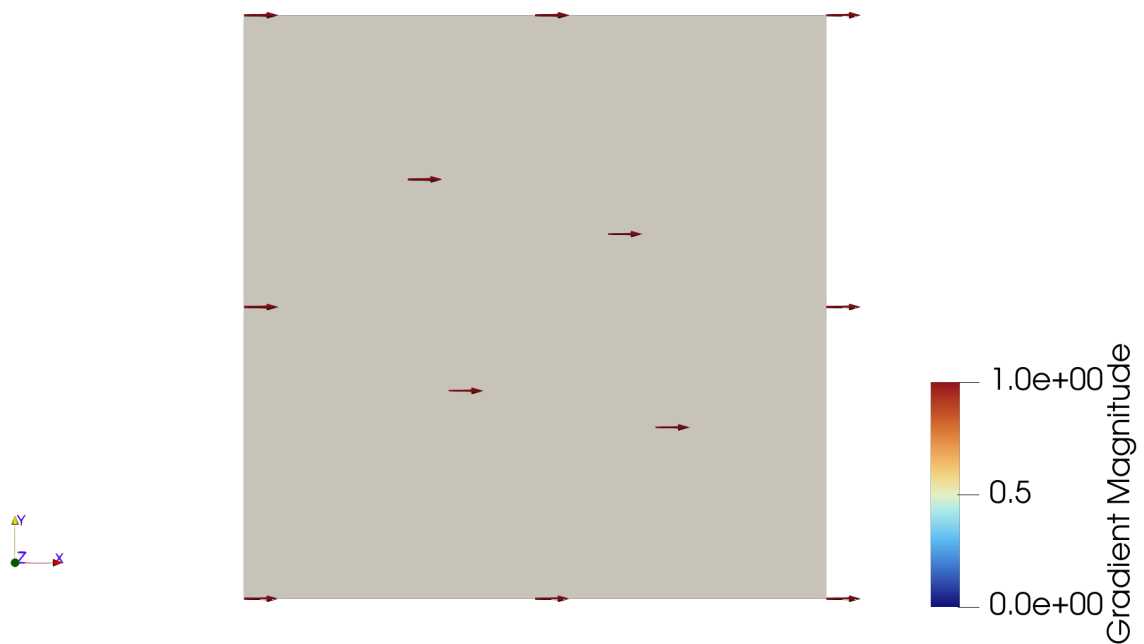


Figura 4: Mapa de gradiente con $f(x, y) = x$

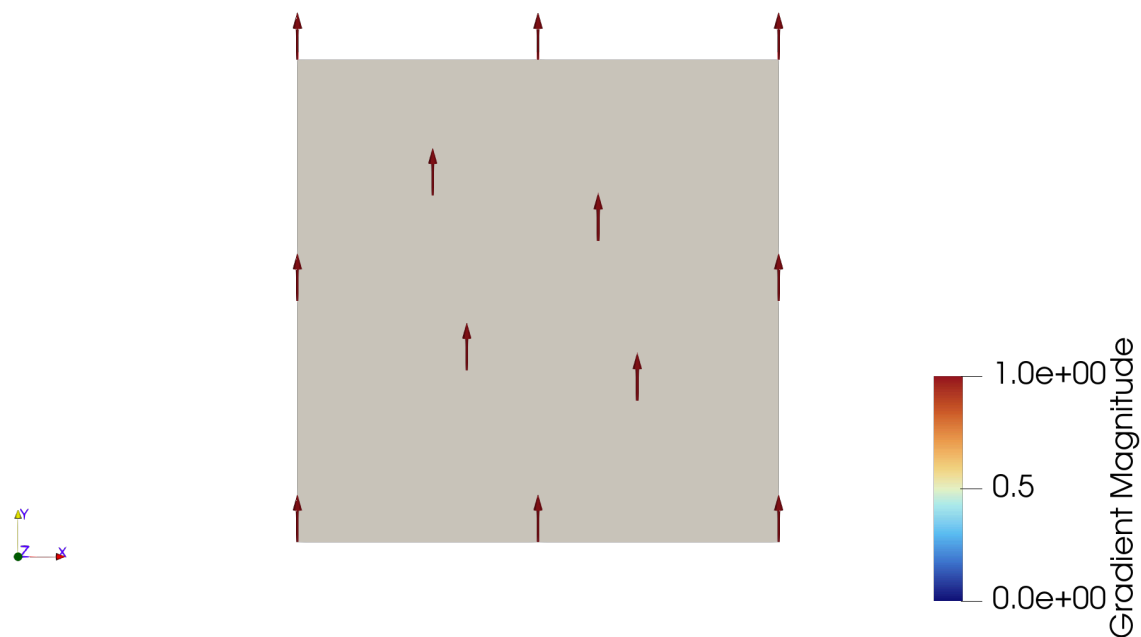


Figura 5: Mapa de gradiente con $f(x, y) = y$

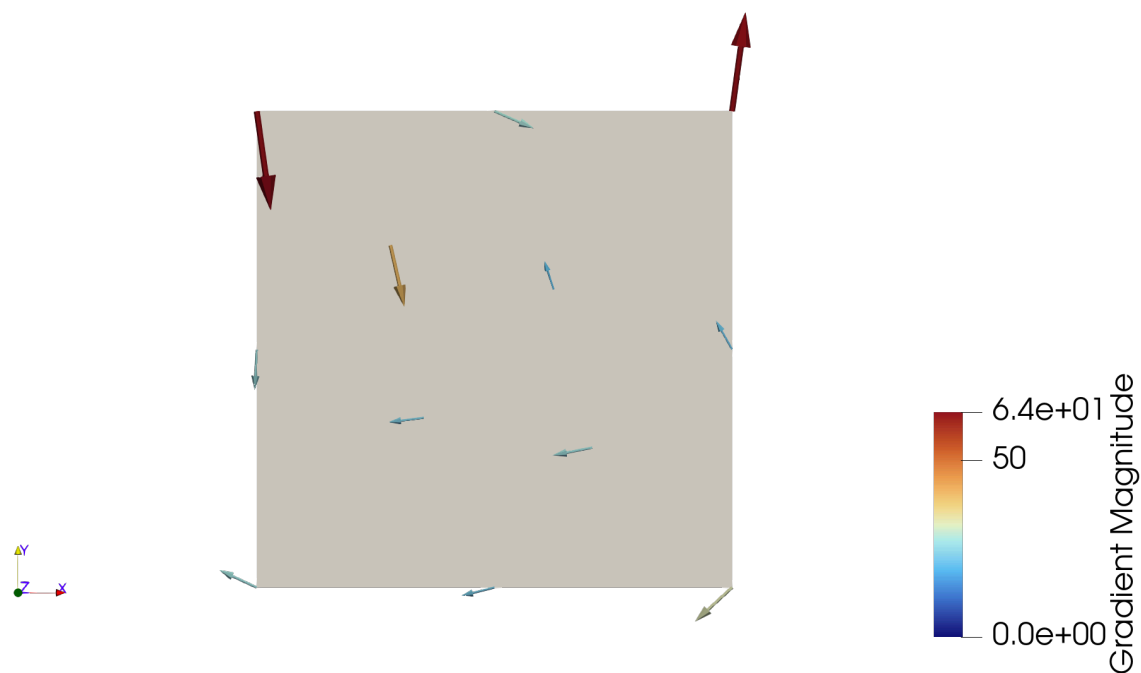
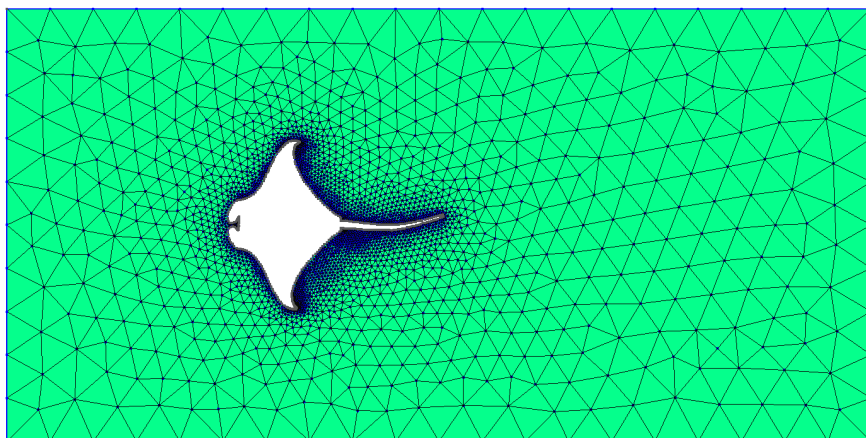


Figura 6: Mapa de gradiente con $f(x, y) = 20(\sin(4x) \sin(3y) + 0.3 \cos(6x) \sin(5y) + 1)$

5.2. Ejemplo 2



Y
X

Figura 7: Discretización de superficie con mantarraya.

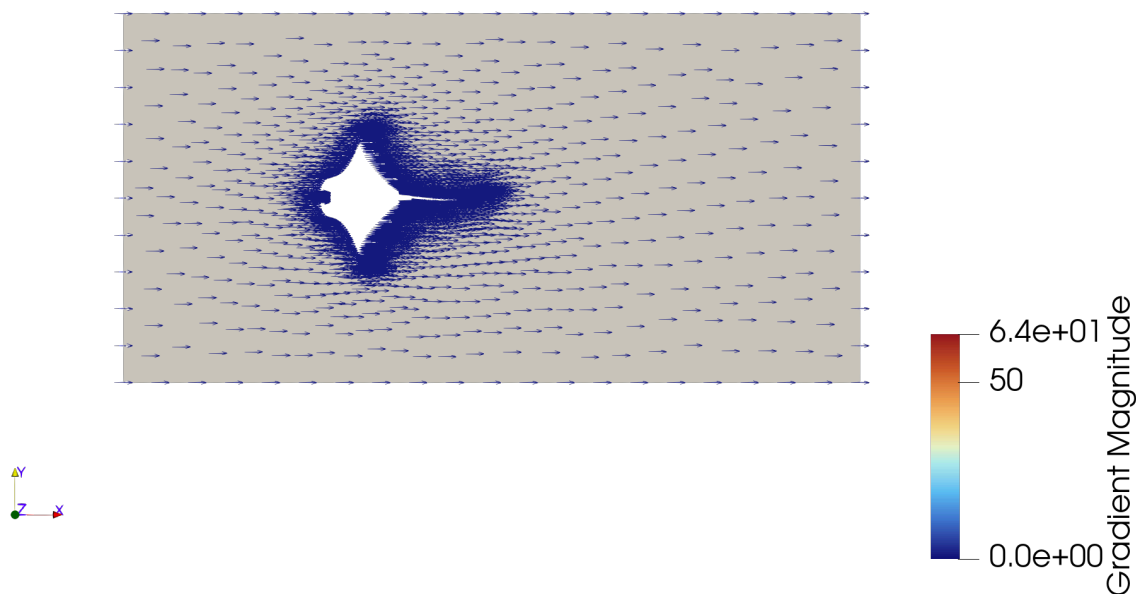


Figura 8: Mapa de gradiente con $f(x, y) = x$

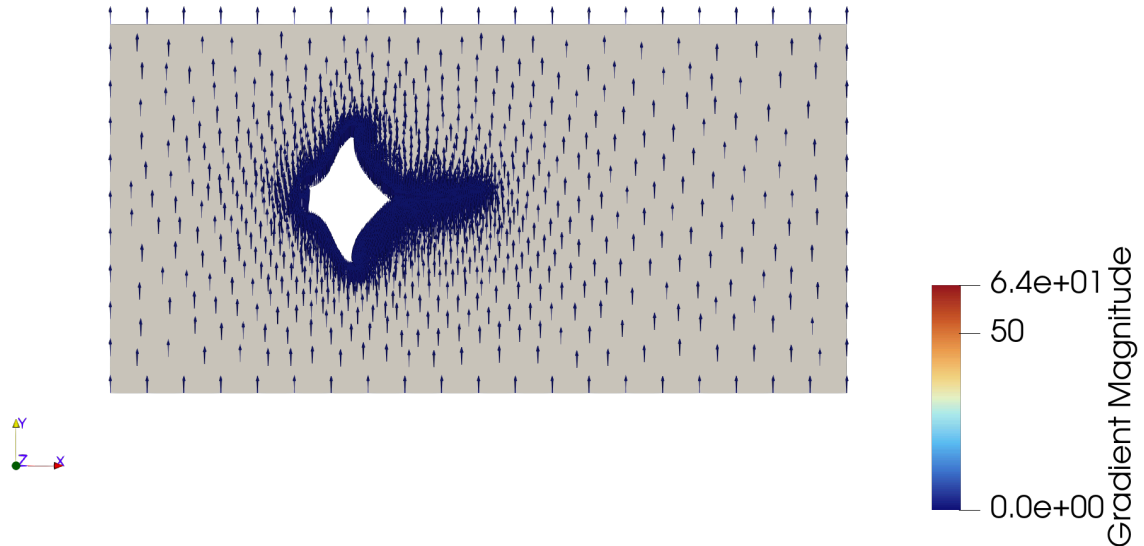


Figura 9: Mapa de gradiente con $f(x, y) = y$

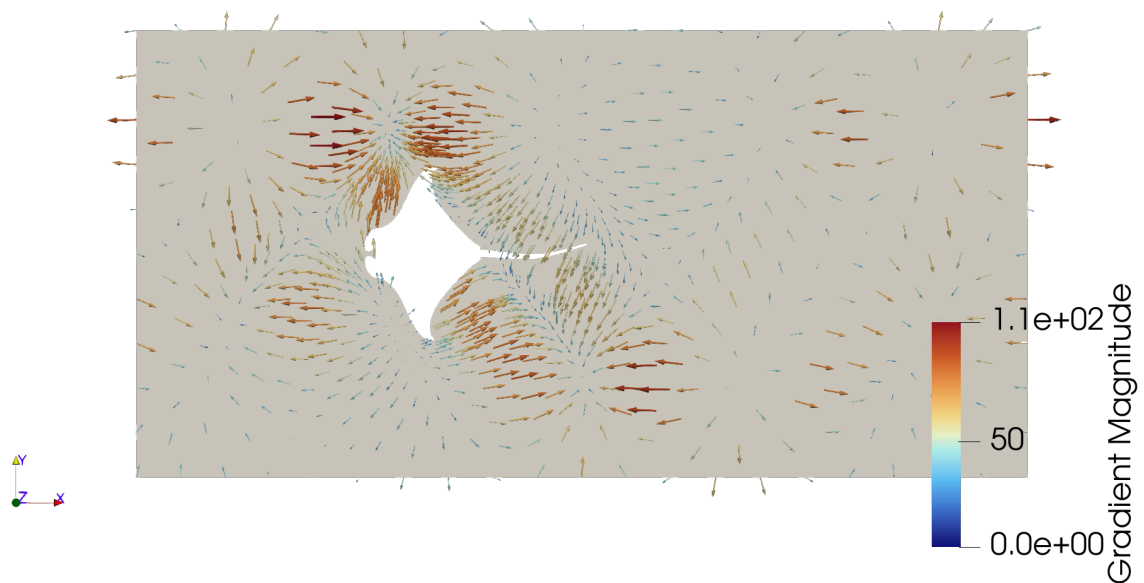


Figura 10: Mapa de gradiente con $f(x, y) = 20(\sin(4x) \sin(3y) + 0.3 \cos(6x) \sin(5y) + 1)$. No se incluyeron todos los puntos, ya que se pierde claridad.

6. Compilación y ejecución

Para compilar, creamos un directorio:

```
mkdir build && cd build
```

Luego, realizamos la build

```
cmake --build . -j4
```

Esto nos genera 2 ejecutables:

```
MeshCalculusOpenMPI  
MeshCalculusSequential
```

Para ejecutar la versión secuencial, usamos

```
./MeshCalculusSequential ../Malla_Mantarraya.vtk mantaralla_seq
```

donde el primer argumento representa el archivo vtk a usar, y el segundo el nombre del archivo al que escribiremos el valor de los gradientes. El archivo es creado en la carpeta output. Las figuras anteriores son resultado de visualizarlas en ParaView.

7. Comparación de tiempos de ejecución

Los siguientes tiempos de ejecución son usando optimización O3.

Versión	Tiempo (s)	Speedup	Eficiencia
Secuencial	0.00256204	1.0	1.0
OpenMPI (-p 2)	0.0015583	1.646	0.823
OpenMPI (-p 4)	0.001466	1.74	0.43
OpenMPI (-p 8)	0.0023493	1.09	0.13
OpenMPI (-p 16)	0.0561549	0.04	0.002

Tabla 1: Tiempos de ejecución con una malla con parámetros $tm_E = 0.20$; $tm_I = 0.02$

Versión	Tiempo (s)	Speedup	Eficiencia
Secuencial	0.099574	1.0	1.0
OpenMPI (-p 2)	0.1138762	0.87	0.43
OpenMPI (-p 4)	0.0923903	1.07	0.26
OpenMPI (-p 8)	0.090198	1.1	0.13
OpenMPI (-p 16)	1.86538	0.05	0.003

Tabla 2: Tiempos de ejecución con una malla con parámetros $tm_E = 0.01$; $tm_I = 0.01$ (malla más fina)

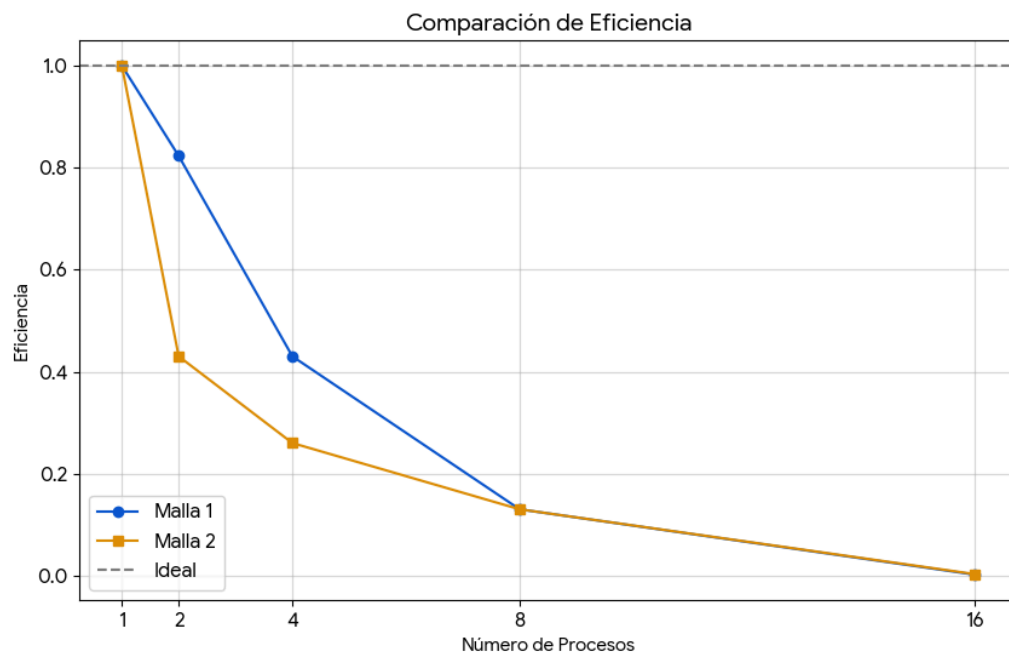


Figura 11: Gráfica de eficiencia entre ejecuciones.

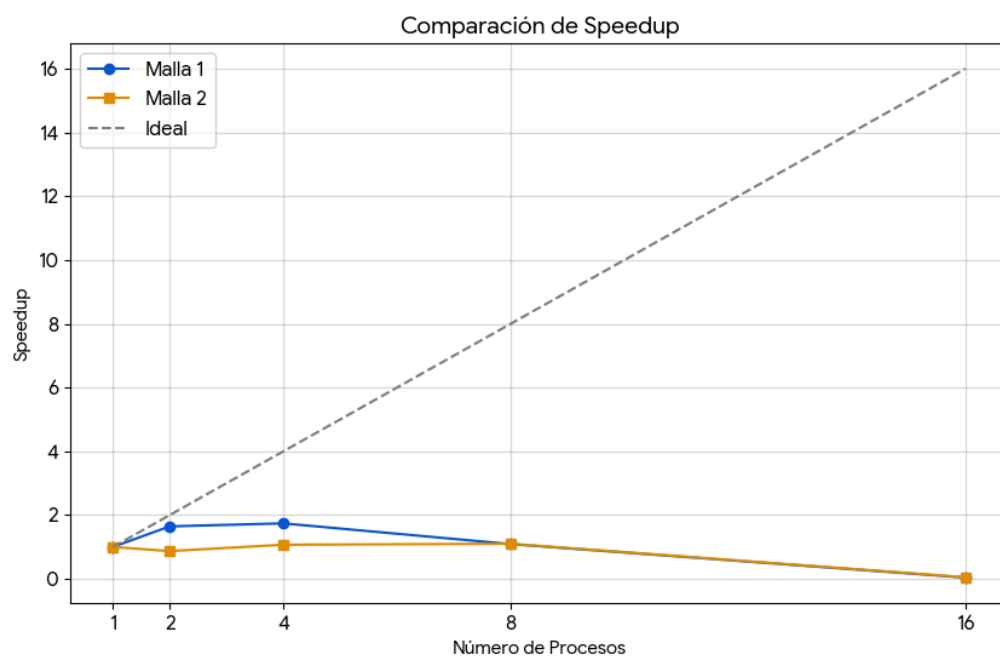


Figura 12: Gráfica de speedup entre ejecuciones.

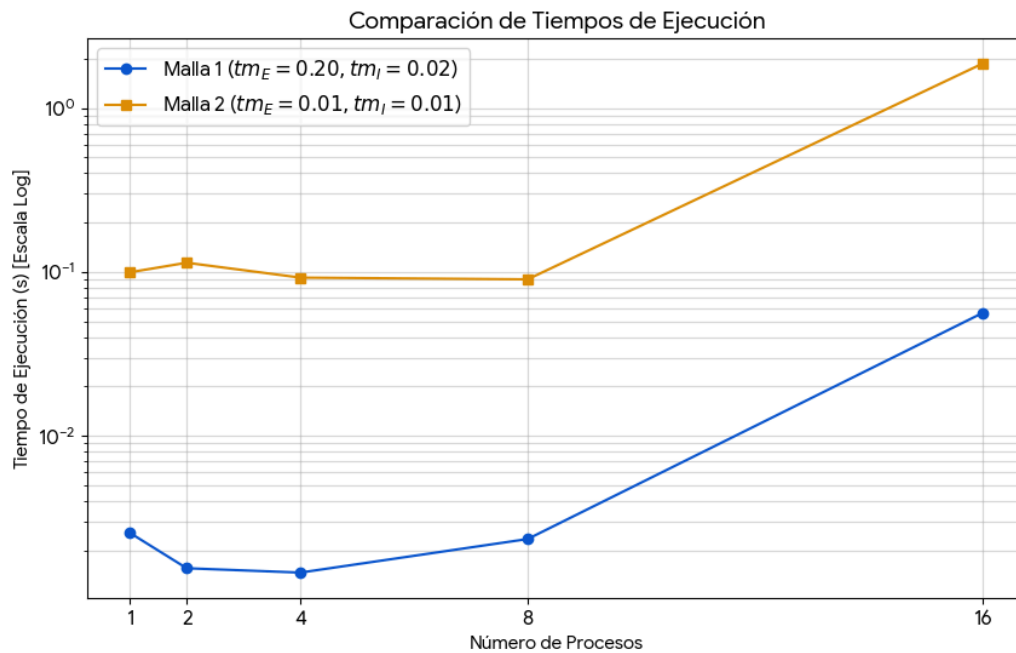


Figura 13: Gráfica de tiempos de ejecución.

8. Conclusiones

La implementación paralela logra vencer a la secuencial por factores mínimos para 2, 4 y 8 procesos. Se deteriora el speedup para 16 procesadores. En la malla más fina (donde hay más celdas), la implementación paralela decae en eficiencia. Esto nos dice que la optimización del compilador con O3 permite realizar cálculos a una velocidad que hace parecer la comunicación entre procesos demasiada. Dado que la comunicación que se debe hacer es grande debido al número de celdas, esto tiene sentido. Idealmente, buscaríamos una eficiencia de 1 y un speedup lineal respecto a los procesos utilizados.